The Core Problem: Searching is Slow

Imagine you have a list of millions of user profiles and you want to find "Alice".

- If you use a Linked List, you have to check every single node until you find her (O(n) time).
- If you use an Array, you still have to check every slot unless you know exactly which index Alice is stored at.

The only data structure that gives us true instant O(1) access is an Array,
but only if we know the exact integer index.

A Hash Table is essentially a clever bridge that converts a search key (like the string "Alice")
into an integer array index so we can use that instant array access.

The Bridge: The Hash Function

To bridge the gap between a key ("Alice") and an array index (e.g., 4), we use a Hash Function.

A hash function takes an input of any size (a string, an object, a number) and scrambles it into a fixed-size integer.

A good hash function must have three properties:

1. Deterministic: If you feed it "Alice", it must output the exact same number every single time.
2. Uniform Distribution: It should spread the outputs evenly. If it maps every name to the number 5, it defeats the purpose.
3. Fast to Compute: If calculating the hash takes longer than searching a list, we've lost our performance advantage.

The Modulo Operation:

Hash functions often generate massive integers.
To make sure this integer fits into our underlying array, we use the modulo operator (%).

index = hash("Alice") % array_size
If our array has 10 slots, and the hash of "Alice" is 104, 104 % 10 = 4. Alice goes into index 4.

The Inevitable Issue: Collisions

By the Pigeonhole Principle, if you have an infinite number of possible strings
but a finite array size (say, 10 slots), eventually, two different keys will map to the exact same index.

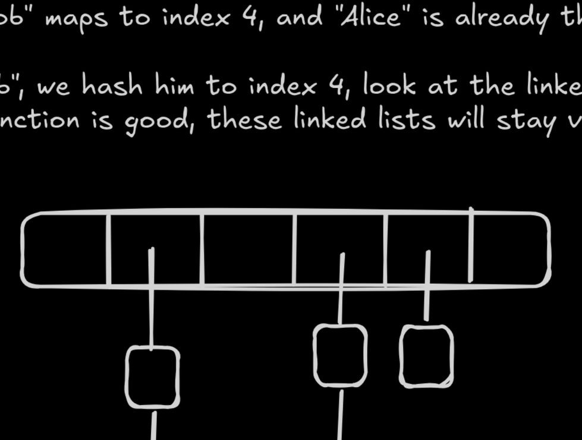
For example, "Alice" might map to index 4, and "Bob" might also map to index 4.
This is called a Collision. Hash tables must have a strategy to resolve this.

Strategy A: Separate Chaining

Instead of storing the data directly in the array slot, each array slot holds a pointer to a Linked List (a "chain").

How it works: When "Bob" maps to index 4, and "Alice" is already there, we just add "Bob" to the linked list at index 4.

Searching: To find "Bob", we hash him to index 4, look at the linked list there, and quickly scan through it.
As long as our hash function is good, these linked lists will stay very short, keeping our search time near O(1).



Strategy B: Open Addressing

In open addressing, all elements are stored inside the array itself.

No linked lists.
No external buckets.

If collision happens → we search for another empty slot inside the table.

What Is Probing?

Probing = the sequence of indices we try after a collision.

Linear Probing

How it works: If "Bob" maps to index 4, but "Alice" is already sitting there, the hash table says:
"Index 4 is taken. Let's look at index 5. Is it empty? Yes? Okay, Bob goes in index 5."

Searching: To find "Bob", we check index 4. We see "Alice".
We know a collision might have happened, so we step forward to index 5 and find "Bob".
(There are other probing methods, like quadratic probing, to prevent elements from bunching up).

```
h(key, i) = (h(key) + i) % m
```

Quadratic Probing

```
h(key, i) = (h(key) + i^2) % m
```

Double Hashing

```
h(key, i) = (h1(key) + i * h2(key)) % m
```

The Bottleneck: The CPU is Starving for Data

Modern CPUs are incredibly fast, but main memory (RAM) is comparatively very slow.
If the CPU had to wait for RAM every time it needed a piece of data, it would spend
most of its time doing absolutely nothing.

To fix this, CPUs have small, ultra-fast memory banks built directly into the chip called
Caches (L1, L2, and L3).

The Golden Rule of Caching (Spatial Locality): When the CPU asks for a piece of data
at a specific memory address, it doesn't just pull that single byte from RAM.
It assumes that if you need the data at address X, you will probably need the data
at X+1, X+2, etc., very soon.

So, the CPU pulls a whole chunk of contiguous memory from RAM into the fast cache all at once.
This chunk is called a Cache Line (typically 64 bytes).If the CPU needs data and it is already
in the cache, that is a Cache Hit (blazing fast). If the data is not in the cache,
it is a Cache Miss (the CPU stops and waits hundreds of cycles for RAM to fetch the next chunk).

Why Separate Chaining is a "Cache Nightmare"

In separate chaining, our hash table is an array of pointers, and each pointer points to the head of a Linked List.

Every time you add a new item to a linked list, the operating system allocates memory for that node
dynamically (using something like malloc in C or new in Java/C++).
Because these nodes are created at different times, they are scattered completely
randomly across the "heap" (your RAM).

What happens during a lookup with chaining:

1. Hash the key, find the array index. (1 memory read)

2. Read the pointer at that index to find the first Linked List node.
   The CPU pulls a 64-byte cache line from RAM containing that node.

3. You check the node. It's not the key you want. You read the next pointer.

4. The next pointer points to an entirely different, random location in RAM.

5. CACHE MISS. The CPU stalls. It has to go all the way out to RAM to fetch a brand new cache line
   for the next node.

6. Repeat until the key is found.

Traversing a linked list is essentially a game of "Pointer Chasing." Because the nodes are not stored next
to each other in memory, the CPU's clever trick of pulling 64-byte chunks is completely wasted.
Every step down the chain is a massive performance penalty.

Why Linear Probing is a "Cache Hero"

In open addressing with linear probing, there are no linked lists and no pointers.
The hash table is just one massive, contiguous block of memory (a single array).

What happens during a lookup with linear probing:

1. Hash the key, find the array index (e.g., index 10).

2. The CPU goes to RAM to check index 10. It pulls a 64-byte cache line.
   Depending on the size of your data, this single cache line might contain the
   slots for index 10, 11, 12, 13, and 14 all at once.

3. You check index 10. It's a collision (the wrong key is there).

4. Because it's linear probing, your next step is to check index 11.

5. CACHE HIT. Index 11 was already pulled into the ultra-fast L1 cache during step 2! The CPU checks it instantly.

6. Index 11 is also a collision? Check 12. CACHE HIT.

With linear probing, resolving a collision by stepping to the next array slot is practically free because of spatial locality.
The CPU is working exactly as it was designed to.

Why don't we always use Linear Probing?

If linear probing is so fast, why does chaining even exist?

Linear probing suffers from a problem called Primary Clustering.
As collisions happen, clumps of contiguous data start to form in the array.
If a new key hashes anywhere near a clump, it gets tacked onto the end of the clump, making
the clump even bigger. Over time, these massive clusters cause lookups to degrade from
O(1) closer to O(n), negating the cache benefits.

Chaining avoids clustering entirely, making its performance much more predictable and stable,
even if the absolute speed of each operation is slightly slower due to cache misses.

Keeping it Fast: The Load Factor and Rehashing

If we keep adding data, our hash table will get full.

- In separate chaining, the linked lists get too long.

- In open addressing, we run out of empty slots, causing long probing searches.

To measure how full the table is, we use the Load Factor:

Load Factor = (Number of Elements) / (Total Array Size)

If the load factor exceeds a certain threshold (commonly 0.7 or 70% full), the hash table does
something drastic to maintain its O(1) speed: Dynamic Resizing (Rehashing).

1. It creates a brand new, larger array (usually double the size).

2. It takes every single element from the old array, recalculates their hash with the
   new array size (hash(key) % new_array_size), and inserts them into the new array.

3. The old array is discarded.

Rehashing is a slow O(n) operation, but because it happens so rarely, the average (amortized)
time to insert or search remains O(1).

Why do we double the size? (Amortized Time)

The Naive Approach: Adding a fixed amount

Imagine we resize by adding exactly 10 new slots every time the array fills up.

- You insert 10 items. (Fast)
- Array is full. You create an array of size 20, copy 10 items.
- You insert 10 more items. (Fast)
- Array is full. You create an array of size 30, copy 20 items.
- Array is full. You create an array of size 40, copy 30 items.

If you insert N items, the number of copies you do grows linearly every time.
The total work you do copying elements is $10 + 20 + 30 + ... + N, which mathematically
results in an O(N^2) time complexity.

This completely destroys the speed of our hash table.

The Doubling Approach: Multiplying by a constant factor

Now, imagine we double the size every time.

- Array size 1 is full -> resize to 2 (1 copy)
- Array size 2 is full -> resize to 4 (2 copies)
- Array size 4 is full -> resize to 8 (4 copies)
- Array size 8 is full -> resize to 16 (8 copies)

If we have just reached a size of N, the total number of copies we
have performed over the entire lifetime of the hash table is a geometric series:

1 + 2 + 4 + 8 + 16 + ... + N / 2

The sum of this entire series is strictly less than N.
This means if you insert N items, you do at most N copy operations.

Because N insertions + N copies = 2N operations, the average work done per insertion is just 2N/N = 2.
Constant time! By doubling, we guarantee that the painful O(N) resize happens exponentially less often as
the table grows, keeping the average insert time strictly to Amortized O(1).

Why a Power of 2?

In our breakdown, we determined the array index using the modulo operator:

index = hash(key) % array_size

Here is the problem: division and modulo are the slowest arithmetic operations a CPU can do.
While addition or bitwise operations might take 1 CPU cycle, a modulo operation can take anywhere
from 10 to 30+ cycles. If you are doing millions of lookups, that bottleneck becomes massive.

However, if your array_size is strictly a power of 2 (2, 4, 8, 16, 32, 64...),
you can use a bitwise cheat code to completely skip the slow modulo operation.

The Bitwise AND trick

When dividing by a power of 2, the remainder is simply the lowest bits of the number.
CPUs have an operation called Bitwise AND (&) that can extract these bits in a single, blistering-fast cycle.

Mathematically, if array_size is a power of 2, then:

$$hash \pmod{array\_size} \equiv hash \,\&\, (array\_size - 1)$$

Let's look at it in Binary:

Assume our array_size is 16 (which is 2^4).
We want to find hash % 16.Notice that 16 - 1 = 15.
In binary, 15 is 0000 1111.

Now, let's say our hash function spits out the number 43.
In binary, 43 is 0010 1011.

Watch what happens when we do 43 & 15:

```
   0010 1011  (Hash: 43)
& 0000 1111  (Size - 1: 15)
-------------------
  0000 1011  (Result: 11)
```

The bitwise & acts like a mask. Because 15 is just a solid block of 1s at the end,
it zeroes out all the higher bits of the hash and perfectly slices off the lowest 4 bits.
43 (mod 16) is indeed 11.

By forcing the underlying array to always be a power of 2,
modern hash tables (like Java's HashMap or Python's dict) can replace an expensive 20-cycle division
operation with a 1-cycle bitwise operation for every single insert, lookup, and deletion.